



Implementing EKF on a nonlinear model

- Refactor EKF code: represent real-time implementation better.
 - Initialization routine (`initEKF.m`), called once at startup.
 - Update routine (`iterEKF.m`), called every sample interval.
 - “Wrapper” code, coordinates the entire simulation process.
- Wrapper code begins (load data, store in local variables):

```
load simOut.mat model dT t u x z % Load model parameters and simulation profile

[nx,nt] = size(x); % Reserve storage for computed
xhatstore = zeros(nx,nt); % state estimates and bounds,
boundstore = zeros(nx,nt); % for plotting

SigmaX0 = 0.1*eye(nx,nx); % Define uncertainty of initial state
xhat0 = zeros(nx,1); % Define expected value of initial state

% Create ekfData structure and initialize variables
ekfData = initEKF(xhat0,SigmaX0,u(1),model);
```



Wrapper code (2): Main loop

- Wrapper code continues by entering main simulation loop.
 - “Measure” sensor readings; update EKF.

```
% Now, enter loop for remainder of time, where we update the EKF
% once per sample interval
hwait = waitbar(0,'Computing...');
for k = 2:length(t)
    uk = u(k); % "measure" input force
    zk = z(k); % "measure" nonlinear output

    % Compute estimate of present state vector and its bounds
    [xhatstore(:,k),boundstore(:,k),ekfData] = iterEKF(uk,zk,dT,ekfData);

    % update waitbar periodically, but not too often (slow procedure)
    if mod(k,100) == 0
        waitbar(k/length(t),hwait);
    end
end
close(hwait);
```



Wrapper code (3): Plot state and estimate

- Wrapper code continues with plotting code.
- First, we plot the states, estimates, and confidence bounds.

```
% Plot state and estimate, with bounds
figure(1); clf; CL = lines;
t2 = [t fliplr(t)]; % Prepare for plotting bounds via "fill"
x2 = [xhatstore-boundstore fliplr(xhatstore+boundstore)];
h1 = fill(t2,x2,CL(1,:), 'FaceAlpha',0.5, 'LineStyle','none'); hold on; grid on
set(gca, 'ColorOrderIndex',1);
h2 = plot(t,x',t,xhatstore,'--'); ylim([-0.2 0.25]);
legend([h2;h1(1)],{'True position','True velocity','Position estimate',...
    'Velocity estimate','Bounds'},'NumColumns',3);
title('Demonstration of EKF filter state estimates');
xlabel('Time (s)'); ylabel('State (m or m/s)');
```



Wrapper code (4): Plot errors

- Next, we plot estimation error and output some statistics.

```
% Plot estimation error with confidence bounds.
figure(2); clf
for k = 1:nx
    subplot(nx,1,k);
    fill(t2,[-boundstore(k,:) flipplr(boundstore(k,:))], 'b',...
        'FaceAlpha',0.1,'LineStyle','none'); hold on; grid on;
    plot(t,xerr(k,:), 'b');
    title(sprintf('State %d estimation error with bounds',k));
    hx = xlabel('Time (s)'); ylabel('Error');
end

% Compute and output some statistics
ekfErr = x - xhatstore;
ekfBnd = boundstore;
fprintf('RMS state estimation error = %g\n',rms(ekfErr(:)));
ind = find(abs(ekfErr)>ekfBnd);
fprintf('Time error outside bounds = %g%\n',length(ind)/numel(x)*100);
```



EKF initialization code

- The EKF initialization code sets up a data structure named `ekfData` that is updated recursively in the main loop.

```
function ekfData = initEKF(xhat0,SigmaX0,priorU,model)
    ekfData.xhat = xhat0; % Initial state description
    ekfData.SigmaX = SigmaX0; % Initial estimation-error covariance
    ekfData.SigmaV = model.SigmaV; % Sensor-noise covariance
    ekfData.SigmaW = model.SigmaW; % Process-noise covariance using "little trick"
    ekfData.Sw = model.Sw; % Cts-time process-noise covariance
    ekfData.priorU = priorU; % Previous value of input force
    ekfData.model = model; % Store model data structure too
    ekfData.Qbump = 5; % In case SigmaX collapses
end
```



Summary

- Implementation of EKF on nonlinear model refactors code.
 - Initialization routine (`initEKF.m`), called once at startup.
 - Update routine (`iterEKF.m`), called every sample interval.
 - “Wrapper” code, coordinates the entire simulation process.
- You have now seen the details of the initialization routine and the wrapper code.
- Next lesson will present the update routine, `iterEKF.m`.